# GROEIPROJECT
# MODULE 7: XML & JSON

## 1. Voorbereiding

1.1. Open het groeiproject en maak daarin een nieuwe module: `m7_xml_json`

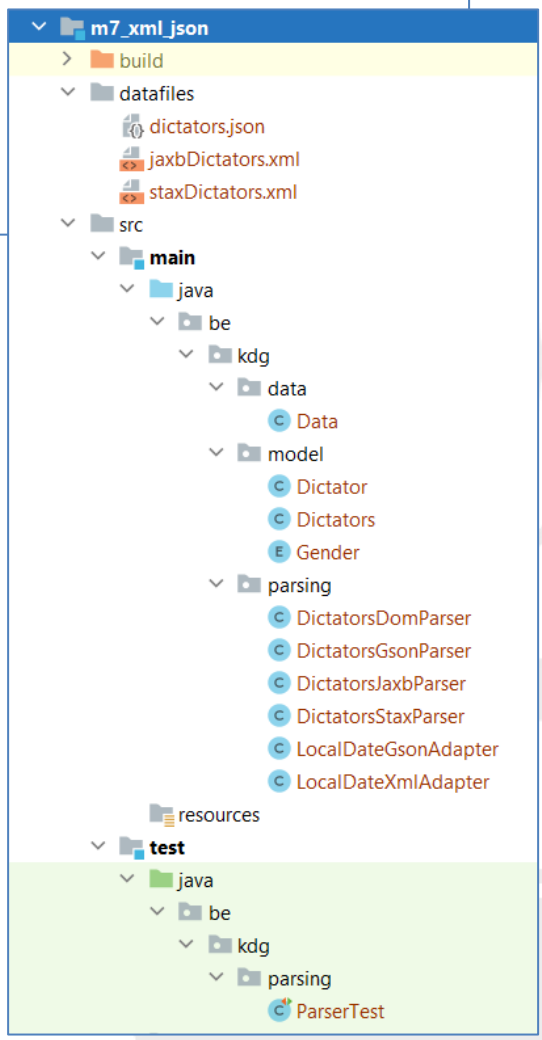
1.2. Kopieer van module `1_herhaling` de package `model` (met o.a. de oorspronkelijke basis- en multiklasse). Kopieer ook de hele package `data`.

1.3. Pas de `build.gradle` van deze module aan: er is deze keer GEEN `mainClassName`, maar je moet wel de dependencies aanpassen om **junit, jaxb en gson** te kunnen draaien: (**opgelet** met copy/paste vanuit deze pdf: sommige karakters (zoals `'-'`) worden genegeerd)

```
dependencies {
    testImplementation platform('org.junit:junit-bom:5.10.0')
    testImplementation 'org.junit.jupiter:junit-jupiter'
    implementation 'org.glassfish.jaxb:jaxb-runtime:4.0.6'
    implementation 'jakarta.xml.bind:jakarta.xml.bind-api:4.0.4'
    implementation 'com.google.code.gson:gson:2.13.2'
}

test{
    useJUnitPlatform()
    testLogging {
        showStandardStreams = true
    }
    ignoreFailures=true
}
```

1.4. Maak in de module `m7_xml_json` een nieuwe directory (dus **NAAST** src) met de naam **datafiles**. Hierin komen de xml- en json-bestanden te staan. Hiernaast een screenshot van de gewenste projectstructuur als je alle opdrachten van deze week afgewerkt hebt →






## 2. XML

2.1. We gaan het parsen naar XML eerst uitproberen met **StAX** technologie.
Maak een nieuwe package met de naam: `parsing` en daarin een klasse met de naam: `XxxStaxParser` (in ons voorbeeld: `DictatorsStaxParser`).

2.2. Voorzie hier 2 private attributen: een object van de multiklasse en een `XMLStreamWriter`.
Voorzie ook een constructor met 2 parameters: de multiklasse en een String met het pad waarnaar we de data gaan wegschrijven (zie 1.4). In de constructor maak je de `XMLStreamWriter` klaar voor gebruik.
Om een mooie layout te bekomen, gebruik je best een `IndentingXMLStreamWriter`.

2.3. Voorzie een methode `staxWriteXML()` die het volledige multiobject via StAX wegschrijft.

a) Vermijd redundante code: schrijf elk element weg via een afzonderlijke private methode `writeElement`:

```
private void writeElement(Dictator dictator)
```

b) Maak gebruik van de **StAX**-technologie om alle eigenschappen van elk basisobject weg te schrijven, voorzien van de juiste start- en end-tag. Eén van de eigenschappen van het basisobject (kies zelf) gebruik je als **xml-attribute** in de start tag van dat element (In ons voorbeeld: het **regime** van de dictator)

c) Denk er ook aan dat je alle waarden moet omzetten naar String (dus ook enum, LocalDate, int, double…)

d) Vergeet de `XMLStreamWriter` niet te sluiten op het einde.

2.4. Maak in de package `parsing` een nieuwe klasse `XxxDomParser` om het xml-bestand met DOM-technologie in te lezen en om te zetten naar Java objecten. (In ons voorbeeld is dat: `DictatorsDomParser`.) Doe dat met de volgende methode:

```
public static Dictators domReadXML(String fileName)
```

a) Maak gebruik van **DOM**-technologie om het xml-bestand element per element in te lezen.

b) Opgelet: alle waarden komen als string binnen en moet je dus nog op gepaste wijze omzetten naar enum, LocalDate, int, double, … (Voor LocalDate kan dat met de methode: LocalDate.parse(string))

c) De returnwaarde is een gevuld multiobject (in ons voorbeeld: `Dictators`)

2.5. Maak onder de testfolder een **testklasse** `be.kdg...parsing.ParserTest`:

a) Schrijf een `@BeforeEach`-methode: vraag aan de klasse `Data` een gevulde List met basisobjecten en maak daarmee een multiobject.

b) Schrijf een testmethode `testStaxDom`. Creëer daar de `XxxStaxParser` (geef het juiste pad mee; bijvoorbeeld "datafiles/staxDictators.xml") en roep de methode `writeXml` op. Roep vervolgens de methode `domReadXml` en vang de returnwaarde op in een nieuw multiobject. Vergelijk het oorspronkelijke en het ingelezen multiobject met elkaar.

c) Controleer of de test groen kleurt.

d) Controleer ook de inhoud van de gecreëerde XML-file (zie volgende pagina).

```xml
<?xml version="1.0" ?>
<dictators>
  <dictator regime="Nazisme">
    <name>Adolf Hitler</name>
    <gender>MAN</gender>
    <birth>1889-04-20</birth>
    <country>Duitsland</country>
    <duration>12</duration>
    <cruelty>100.0</cruelty>
    <victims>66.0</victims>
  </dictator>
  <dictator regime="Militaire dictatuur">
    <name>Augusto Pinochet</name>
    <gender>MAN</gender>
    <birth>1915-11-25</birth>
    <country>Chili</country>
    <duration>17</duration>
    <cruelty>30.0</cruelty>
    <victims>0.12</victims>
  </dictator>
  <dictator regime="Fundamentalisme">
    <name>Ayatollah Khomeini</name>
    <gender>MAN</gender>
```

Eén van de eigenschappen van het basisobject gebruik je als **xml-attribute** (zie 2.3.b)

2.6. We gaan nu automatische binding toepassen via **JAXB**-technologie. Zorg dat de inhoud van het XML bestand dat door JAXB geschreven wordt identiek is als het XML document dat je met StAX schreef. Daartoe moeten we eerst een aantal aanpassingen doen aan onze basisklasse én onze multiklasse:

a) In de **basisklasse** (in ons geval: `Dictator`):

- Gebruik een annotatie om de volgorde van de tags vast te leggen
- Gebruik een annotatie boven een van de setters waar je naar een XML-attribute moet mappen
- Indien er attributen zijn met hoofdletters in het midden, annoteer de methoden dan om ze weg te schrijven volgens de XML conventie (kleine letters, met een koppelteken voor de plaats waar de hoofdletter stond)
- Gebruik `@XmlJavaTypeAdapter` boven elke setter van het type LocalDate. Verwijs daar naar een apart te schrijven klasse `LocalDateXmlAdapter` (zie slides)
- Controleer of er een default constructor aanwezig is (wordt opgroepen door JAXB)

b) In de **multiklasse** (in ons geval: `Dictators`):

- Gebruik `@XmlRootElement`
- Verander de ingekapselde `TreeSet` naar `ArrayList`, pas de code hieraan aan en voorzie een setter en getter naar deze list (in ons voorbeeld: `setDictatorList` en `getDictatorList`). JAXB zal voor ons deze list automatisch vullen.
- Gebruik `@XmlElement` boven de methode `setDictatorList`

2.7. Maak een nieuwe klasse `XxxJaxbParser` (in ons voorbeeld `DictatorsJaxbParser`). Voorzie daar de volgende twee static methoden:

a) `public static void JaxbWriteXml(String file, Object root)`
Maak gebruik van een **Marshaller** om het meegegeven root-object te converteren naar een nieuw XML-bestand.

b) `public static <T> T JaxbReadXml(String file, Class<T> typeParameterClass)`
Maak de methode generiek voor om het even welke multiklasse.
Maak gebruik van een **Unmarshaller** om de data te lezen uit het XML-bestand en te retourneren als gevuld multi-object.

2.8. Voeg een test `testJaxb` toe in de klasse `ParserTest`.
Ga op dezelfde wijze tewerk als in vorige test: schrijf het multiobject weg met JAXB, lees opnieuw in en vergelijk het oorspronkelijke met het ingelezen multiobject.
Controleer of je groen licht krijgt voor beide testen.




## 3. JSON

3.1. Maak een nieuwe klasse `XxxGsonParser` (in ons voorbeeld `DictatorsGsonParser`) en daarin een static methode om de data in JSON-formaat weg te schrijven via **Gson**-technologie.
In ons voorbeeld wordt dat:
`public static void writeJson(Dictators dictators, String fileName)`

3.2. Schrijf ook een methode om de data in JSON-formaat in te lezen via **Gson**-technologie.
In ons voorbeeld wordt dat:
`public static Dictators readJson(String fileName)`

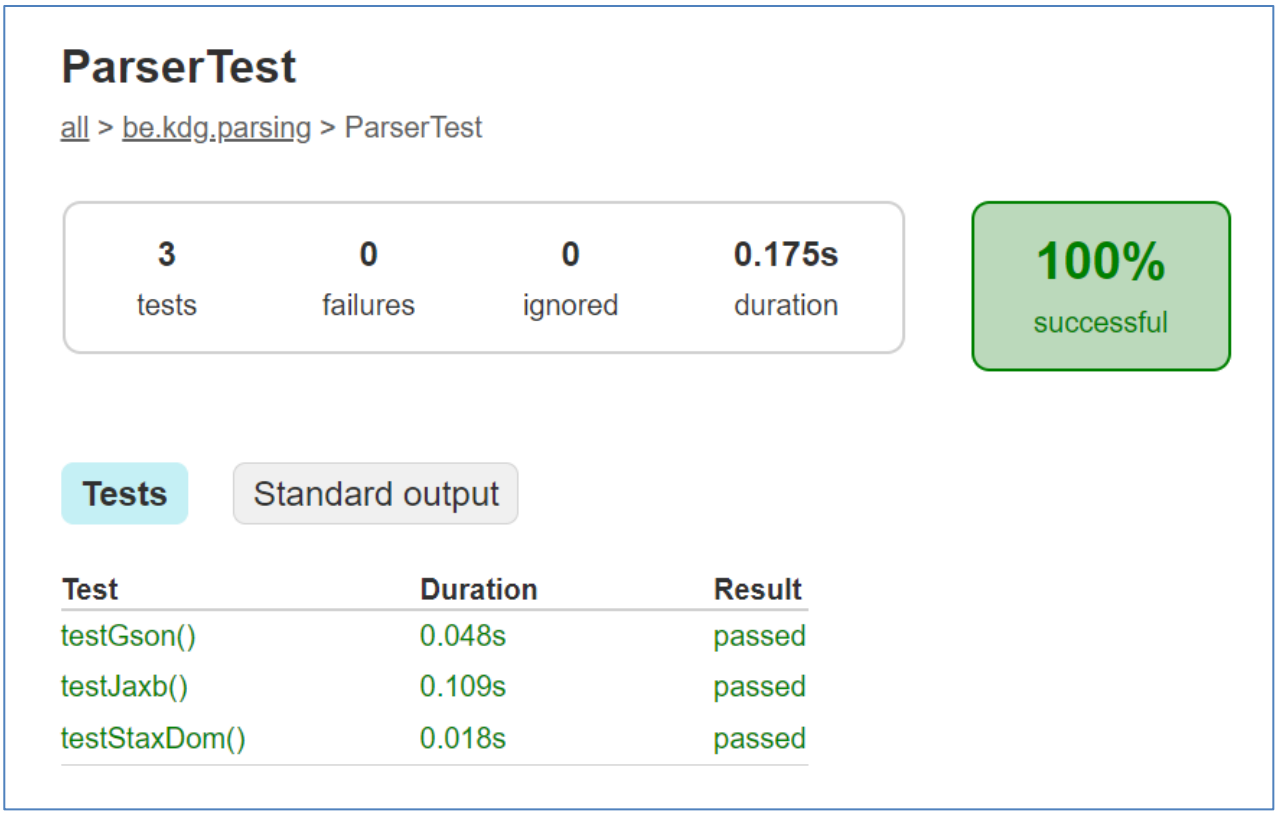
Vergeet ook niet dat je een aparte klasse `LocalDateGsonAdapter` nodig hebt om LocalDate te kunnen schrijven / lezen (zie slides)

3.3. Voeg een testmethode `testGson` toe in `ParserTest` en controleer de correcte werking.
Verifieer ook de JSON-array in het geschreven bestand → let op het gebruik van { of [
- **Links** zie je een voorbeeld waarbij een List van basisobjecten werd weggeschreven
- **Rechts** zie je een voorbeeld waarbij het multiobject werd weggeschreven

```
[
  {
    "name": "Ayatollah Khomeini",
    "gender": "MALE",
    "birth": "1902-09-24",
    "country": "Iran",
    "regime": "Fundamentalisme",
    "duration": 10,
    "cruelty": 99.7,
    "victims": 8.5
  },
  {
    "name": "Nicolae Ceausescu",
    "gender": "MALE",
    "birth": "1918-01-26",
    "country": "Roemenie",
    "regime": "Despotisme",
    "duration": 24,
    "cruelty": 80.0,
    "victims": 0.5
  },
```

```
{
  "dictatorList": [
    {
      "name": "Ayatollah Khomeini",
      "gender": "MALE",
      "birth": "1902-09-24",
      "country": "Iran",
      "regime": "Fundamentalisme",
      "duration": 10,
      "cruelty": 99.7,
      "victims": 8.5
    },
    {
      "name": "Nicolae Ceausescu",
      "gender": "MALE",
      "birth": "1918-01-26",
      "country": "Roemenie",
      "regime": "Despotisme",
      "duration": 24,
      "cruelty": 80.0,
      "victims": 0.5
    },
```

3.4. Controleer of de 3 tests allemaal groen kleuren. Genereer via Gradle een testreport en bestudeer de testresultaten. Welke technologie werkt het snelst: Stax + Dom(1) of JAXB(2) of Gson(3)?



ParserTest

all > be.kdg.parsing > ParserTest

| 3 | 0 | 0 | 0.175s |
|---|---|---|---|
| tests | failures | ignored | duration |

100% successful

Tests | Standard output

| Test | Duration | Result |
|---|---|---|
| testGson() | 0.048s | passed |
| testJaxb() | 0.109s | passed |
| testStaxDom() | 0.018s | passed |

**Tenslotte**: vergeet niet alles te committen naar je repo op GitLab.